



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Rosas Martinez Francisco Javier

N° de Cuenta: 320109766

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 20/Septiembre/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1:

Terminar la Grúa con:

- cuerpo(prisma rectangular)
- base (pirámide cuadrangular)
- 4 llantas(4 cilindros) con teclado se pueden girar las 4 llantas por separado

Para resolver el ejercicio tuve que volver a los datos de posición de la cabina y agregar la base con forma de un prisma rectangular. Después añadí cuatro cilindros (vértices) para representar las ruedas y, por último, puse más teclas para que al presionarlas cada rueda pudiera girar.

Bloques de código generado

Archivo Window.h

```
26 GLfloat getarticulacion7() { return articulacion7; }
27 GLfloat getarticulacion5() { return articulacion5; }
28 GLfloat getarticulacion6() { return articulacion6; }
29 GLfloat getarticulacion7() { return articulacion7; }
30 GLfloat getarticulacion8() { return articulacion8; }
31 GLfloat getarticulacion9() { return articulacion9; }
32 GLfloat getarticulacion10() { return articulacion10; }
33 GLfloat getarticulacion11() { return articulacion11; }
34
35 ~Window();
36 private:
37     GLFWwindow *mainWindow;
38     GLint width, height;
39     GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5,
40     articulacion6, articulacion7, articulacion8, articulacion9, articulacion10, articulacion11;
41     bool keys[1024];
42     GLint bufferWidth, bufferHeight;
43     GLfloat lastX;
44     GLfloat lastY;
45     GLfloat xChange;
46     GLfloat yChange;
47     bool mouseFirstMoved;
```

Archivo Window.cpp

```
12  Window::Window(GLint windowHeight, GLint windowWidth)
13  {
14      width = windowHeight;
15      height = windowWidth;
16      rotax = 0.0f;
17      rotay = 0.0f;
18      rotaz = 0.0f;
19      articulacion1 = 0.0f;
20      articulacion2 = 0.0f;
21      articulacion3 = 0.0f;
22      articulacion4 = 0.0f;
23      articulacion5 = 0.0f;
24      articulacion6 = 0.0f;
25      articulacion7 = 0.0f;
26      articulacion8 = 0.0f;
27      articulacion9 = 0.0f;
28      articulacion10 = 0.0f;
29      articulacion11 = 0.0f;
```

```
159      // Nuevas articulaciones con numeros
160      if (key == GLFW_KEY_1)
161      {
162          theWindow->articulacion7 += 10.0;
163      }
164      if (key == GLFW_KEY_2)
165      {
166          theWindow->articulacion8 += 10.0;
167      }
168      if (key == GLFW_KEY_3)
169      {
170          theWindow->articulacion9 += 10.0;
171      }
172      if (key == GLFW_KEY_4)
173      {
174          theWindow->articulacion10 += 10.0;
175      }
176      if (key == GLFW_KEY_5)
177      {
178          theWindow->articulacion11 += 10.0;
179      }
```

Archivo P04-320109766.cpp

```
//Base
model = model2;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 4.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

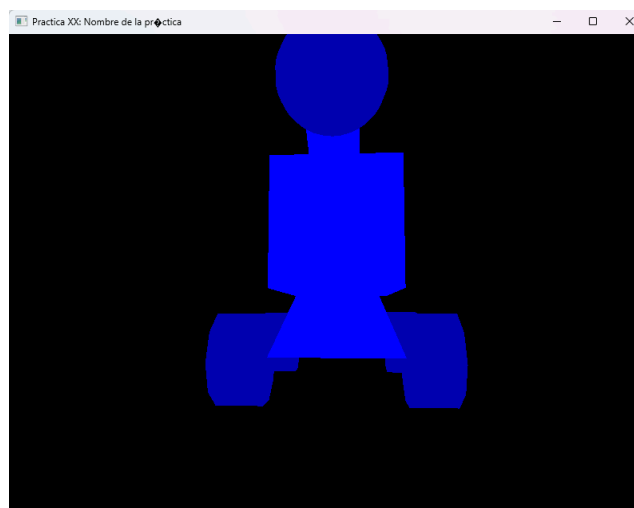
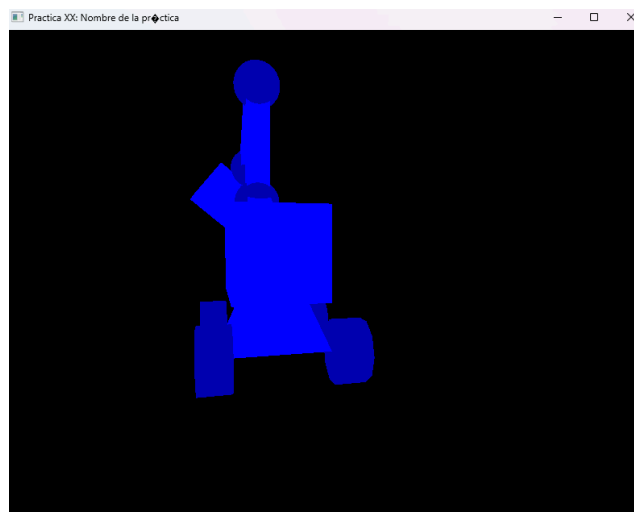
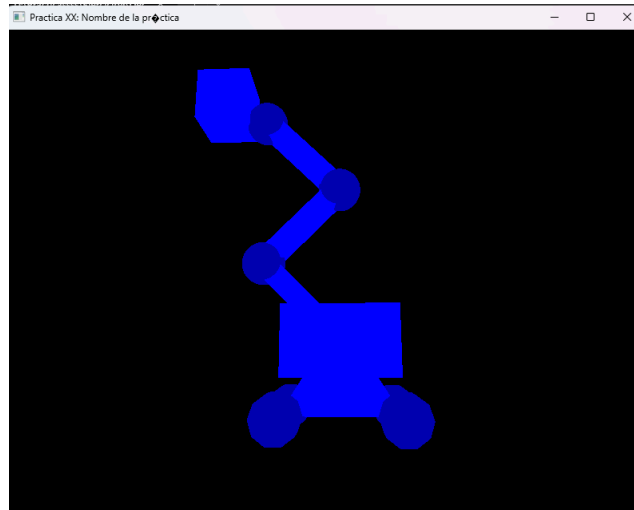
//Llanta 1
model = modelaux;
model = glm::translate(model, glm::vec3(2.5f, -2.0f, 2.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//Llanta 2
model = modelaux;
model = glm::translate(model, glm::vec3(2.5f, -2.0f, -2.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//Llanta 3
model = modelaux;
model = glm::translate(model, glm::vec3(-2.5f, -2.0f, 2.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//Llanta 4
model = modelaux;
model = glm::translate(model, glm::vec3(-2.5f, -2.0f, -2.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();
```

Código en ejecución



Ejercicio 2:

Crear un animal robot 3d

- Instanciando cubos, pirámides, cilindros, conos, esferas:
- 4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)
- cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente)

En este ejercicio primero elegí el cuerpo del robot como elemento principal. Para las piernas seguí un proceso parecido al del brazo de la grúa, pero con el robot cada pierna tiene una rotación de 90 grados respecto al eje Y. Después le añadí una esfera para simular la cabeza, a la que incorporé ojos y dos orejas con movimiento independiente.

Bloques de código generado

```
// CUERPO
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
modelaux = model;
model2 = model;
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
sp.render();

// ARTICULACIÓN PIERNA 1
model = model2;
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// PRIMER BRAZO
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(4.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
```

```

//SEGUNDA ARTICULACIÓN
model = modelaux;
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.2f, 0.2f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// SEGUNDO BRAZO
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

// PIE 1
model = glm::translate(model, glm::vec3(0.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.3f, 0.2f, 1.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.1f, 0.1f, 0.1f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

// ARTICULACIÓN PIERNA 2
model = model2;
model = glm::rotate(model, glm::radians(120.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// PRIMER BRAZO
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

//SEGUNDA ARTICULACIÓN
model = modelaux;
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

```

```

// SEGUNDO BRAZO
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

// PIE 2
model = glm::translate(model, glm::vec3(0.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.3f, 0.2f, 1.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.1f, 0.1f, 0.1f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

// ARTICULACIÓN PIERNA 3
model = model2;
model = glm::rotate(model, glm::radians(210.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// PRIMER BRAZO
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

//SEGUNDA ARTICULACIÓN
model = modelaux;
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// SEGUNDO BRAZO
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

```



```

// PIE 3
model = glm::translate(model, glm::vec3(0.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.3f, 0.2f, 1.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.1f, 0.1f, 0.1f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

// ARTICULACIÓN PIERNA 4
model = model2;
model = glm::rotate(model, glm::radians(300.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// PRIMER BRAZO
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

//SEGUNDA ARTICULACIÓN
model = modelaux;
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// sEGUNDO BRAZO
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(3.5f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

```

```

// PIE 4
model = glm::translate(model, glm::vec3(0.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.3f, 0.2f, 1.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.1f, 0.1f, 0.1f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

// CABEZA
model = model2;
model = glm::translate(model, glm::vec3(0.0f, 3.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, -1.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.7f, 0.7f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

// OJOS
model = glm::rotate(model, glm::radians(-15.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

// OREJA DERECHA
model = modelaux;
model = glm::translate(model, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

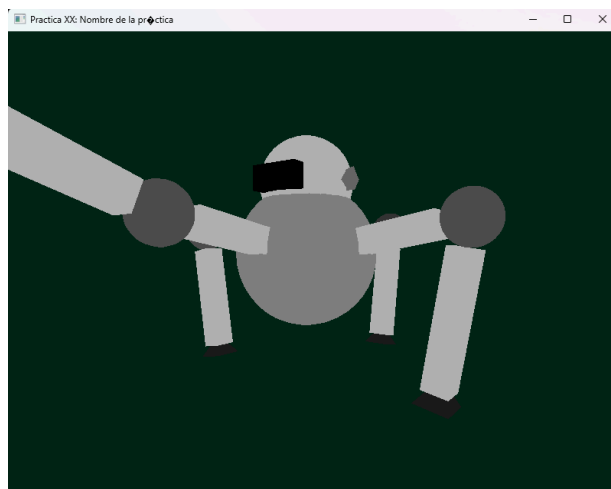
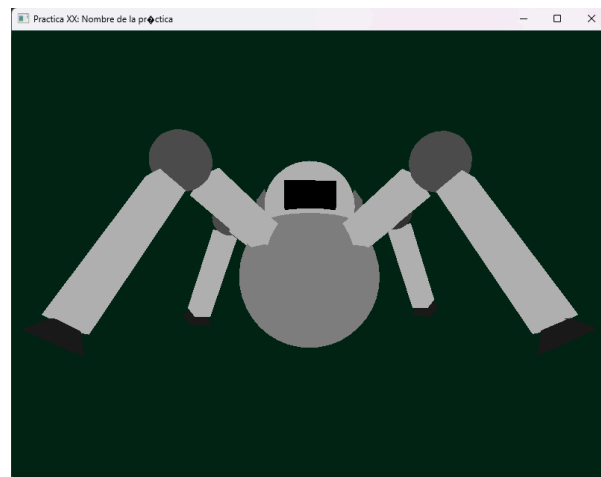
// PIRAMIDE ARRIBA DE OREJA
model = glm::translate(model, glm::vec3(0.0f, 0.9f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();

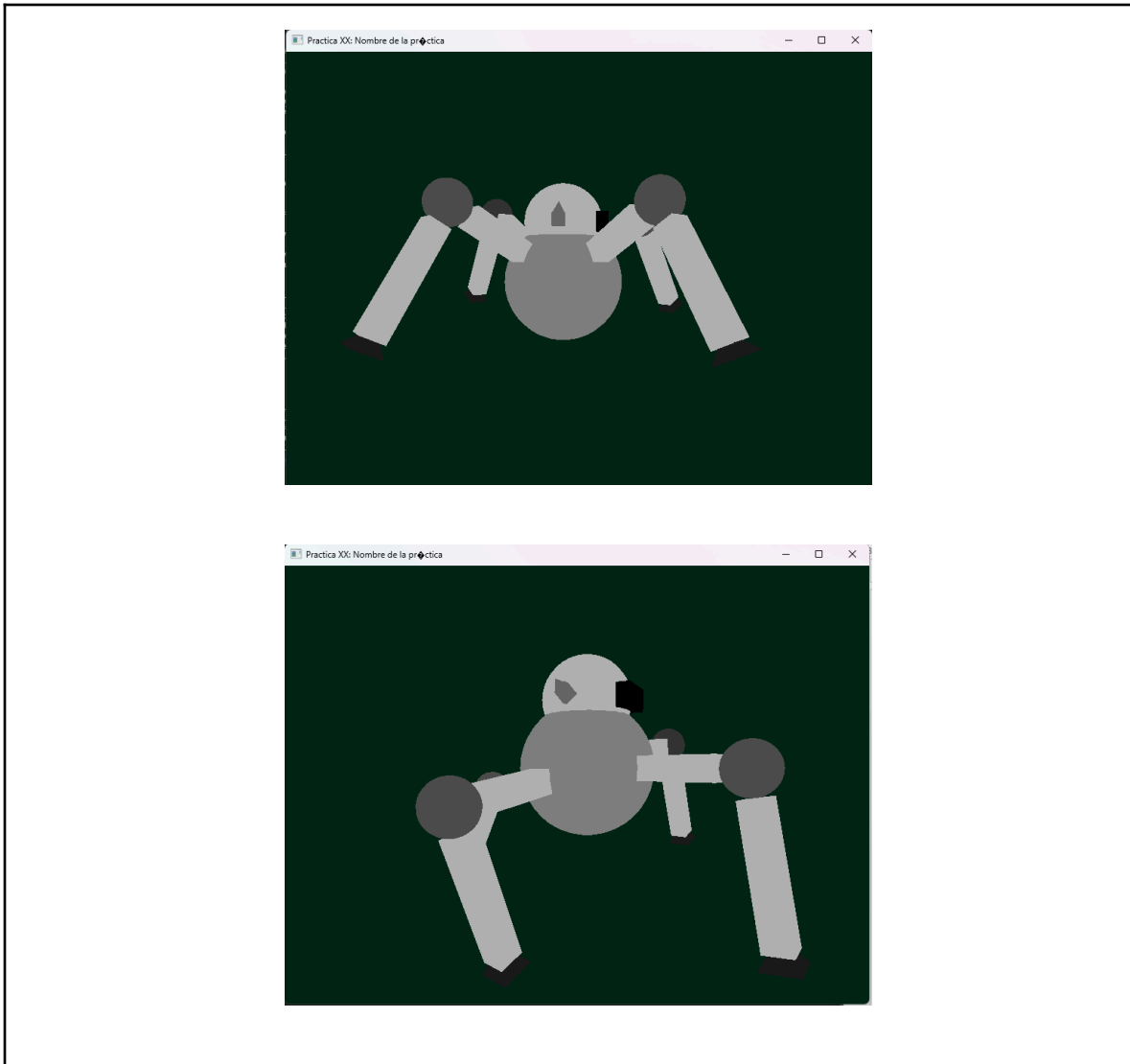
```

```
// OREJA IZQUIERDA
model = modelaux;
model = glm::translate(model, glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(-1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

// PIRAMIDE ARRIBA DE OREJA
model = glm::translate(model, glm::vec3(0.0f, 0.9f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
```

Código en ejecución





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Con esta práctica afortunadamente no tuve ningún inconveniente.

Conclusión:

Al finalizar esta práctica pude comprender y aplicar los conceptos de jerarquía en el modelado gráfico. En el primer ejercicio completé la forma de la grúa agregando una base y cuatro ruedas que giraban de manera independiente con diferentes teclas. En el segundo ejercicio construí desde cero un robot con cuatro piernas que tenían dos articulaciones cada una, además de una cabeza con movimiento y dos orejas móviles a los lados. En general,

esta práctica me resultó muy útil para entender la jerarquía y también bastante entretenida conforme iba armando las figuras.

Bibliografía :

OpenGL Architecture Review Board, Shreiner, D., Woo, M., Neider, J., & Davis, T. (2010). OpenGL Programming Guide: The Official Guide to Learning OpenGL (7th ed.). Addison-Wesley.

OpenGL. (s. f.). OpenGL Documentation. Khronos Group. Recuperado de <https://www.khronos.org/opengl/>

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.